

# SituationReport

## testdata\_generator

### Manuel d'utilisation

Générer des données de tickets Jira synthétiques pour SituationReport

---

`situation-report -- github.com/Jaegerfeld/situation-report`

Pour les Agile Coaches et les PI Managers -- Version 0.26.0

# 1 Qu'est-ce que le Générateur de données de test ?

Le Générateur de données de test (Testdata Generator) crée des données synthétiques de tickets Jira au format Jira REST API. Les fichiers générés peuvent être traités directement par le module `transform_data` — sans instance Jira réelle.

Le générateur simule des historiques de flux de travail réalistes : les tickets parcourent les étapes définies, passent des durées variables dans chaque étape, régressent occasionnellement et se ferment à un taux configurable.

Cas d'utilisation typiques :

- Tester une nouvelle installation : Vérifier que `transform_data` et `build_reports` fonctionnent correctement
- Démonstrations et présentations : Données réalistes sans problèmes de confidentialité
- Formation : Environnement d'apprentissage contrôlé avec des données connues
- Développement : Données de test reproductibles via le paramètre `seed`

Le Générateur complète le module Get Data (en cours de développement), qui charge des données réelles directement depuis Jira. Pour l'export manuel de données Jira réelles, consultez le Manuel d'utilisation Get Data.

## 2 Prérequis et démarrage

---

### 2.1 Package portable (recommandé)

Le package portable inclut Python déjà intégré — aucune installation séparée requise.

| Système d'exploitation | Fichier de démarrage                            |
|------------------------|-------------------------------------------------|
| Windows                | TestdataGenerator.bat (double-clic)             |
| macOS                  | TestdataGenerator.command (clic droit → Ouvrir) |
| Linux                  | ./TestdataGenerator.sh                          |

*Note Windows : Si un message d'erreur concernant Python apparaît, le fichier ZIP a peut-être été bloqué par Windows. Solution : clic droit sur le ZIP → Propriétés → Sécurité → Débloquer → OK → extraire à nouveau.*

### 2.2 Depuis le code source (Python requis)

```
python -m testdata_generator
```

### 2.3 L'interface utilisateur

Au démarrage, la fenêtre principale affiche des champs de saisie pour tous les paramètres. Seul le fichier de flux de travail est obligatoire — tous les autres champs ont des valeurs par défaut sensées.

testdata\_generator 0.9.9

Hilfe

Workflow-Datei

Ausgabedatei (JSON)

Projekt-Key: TEST

Anzahl Issues: 100

Von (YYYY-MM-DD): 2025-01-01

Bis (YYYY-MM-DD): 2025-12-31

Issue-Typen (Typ:Gewicht ...)

Completion-Rate (0-1): 0.7

To-Do-Rate (0-1): 0.15

Backflow-Prob. (0-1): 0.1

Seed (leer = zufällig)

Ø Cycle Time (Tage, leer=auto)

Std.-Abw. (Tage, leer=auto)

Muster: ☒ Kein Muster ☐ Triangle ☐ Flat Triangle ☐ Cluster ☐ Batch

PI-Dauer (Wochen): 12

Generieren

Log

Fig. 1 : Interface utilisateur du Générateur de données de test

## 3 Le fichier de flux de travail

Le fichier de flux de travail définit les étapes de votre tableau Jira et est le seul paramètre obligatoire. Il utilise le même format que `transform_data` — un seul fichier fonctionne pour les deux modules.

### 3.1 Format

Chaque ligne décrit une étape. Les alias (noms de statut alternatifs dans l'export Jira) sont séparés par des deux-points. Deux lignes spéciales marquent le début et la fin du travail actif :

```
NomCanonique:Alias1:Alias2
<First>PremiereEtapeActive
<Closed>EtapeFermee
```

| Ligne                      | Signification                                                           |
|----------------------------|-------------------------------------------------------------------------|
| NomCanonique:Alias1:Alias2 | Étape avec un ou plusieurs noms de statut Jira alternatifs              |
| NomEtape (sans :)          | Étape sans alias                                                        |
| NomEtape                   | Première étape active — où commence le traitement (début du cycle time) |
| NomEtape                   | Étape fermée — marque un ticket comme terminé                           |

### 3.2 Exemple de flux ART\_A

L'exemple fourni `workflow_ART_A.txt` modélise un flux SAFe ART typique.

```
Funnel:New:Open:To Do
Analysis:In Analysis:Estimated
Program Backlog:Ready for Dev:Backlog
Implementation:In Implementation:In Review:In Progress
Blocker
Validating on Staging:QA
Deploying to Production:Ready for Development
Releasing:Completed
Done:Canceled
<First>Analysis
<Closed>Releasing
```

## 4 Référence des paramètres

Tous les paramètres sont disponibles dans l'interface graphique et en ligne de commande.

| Paramètre                | Défaut                             | Description                                                         |
|--------------------------|------------------------------------|---------------------------------------------------------------------|
| --workflow FILE          | (obligatoire)                      | Fichier de définition du flux de travail (.txt)                     |
| --output FILE.json       | _generated.json                    | Chemin du fichier de sortie                                         |
| --project KEY            | TEST                               | Clé de projet Jira pour les tickets générés                         |
| --issues N               | 100                                | Nombre de tickets à générer                                         |
| --from-date YYYY-MM-DD   | 2025-01-01                         | Date de création la plus ancienne                                   |
| --to-date YYYY-MM-DD     | 2025-12-31                         | Date de transition la plus récente                                  |
| --issue-types TYPE:W ... | Feature:0.6 Bug:0.3<br>Enabler:0.1 | Types de tickets avec pondérations                                  |
| --completion-rate FLOAT  | 0.7                                | Fraction de tickets atteignant l'étape fermée (0–1)                 |
| --todo-rate FLOAT        | 0.15                               | Fraction de tickets ouverts restant dans les étapes initiales (0–1) |
| --backflow-prob FLOAT    | 0.1                                | Probabilité d'une transition en arrière à chaque pas (0–1)          |
| --seed INT               | (aléatoire)                        | Seed pour résultat reproductible (optionnel)                        |
| --mean-cycle-days FLOAT  | (aucun)                            | Cycle time moyen en jours (lognormal) ; active le contrôle CT       |
| --std-cycle-days FLOAT   | (30 % de la moyenne)               | Écart type du cycle time en jours                                   |
| --pattern MOTIF          | none                               | Anti-pattern : none / triangle / flat_triangle / cluster / batch    |
| --pi-duration-weeks INT  | 12                                 | Durée du cycle PI en semaines (pour cluster/batch)                  |
| --pattern-strength 0-100 | 50                                 | Intensité du motif 0–100 (0=subtil, 50=défaut, 100=fort)            |

## 4b Modes d'anti-pattern de flux

Depuis la version 0.9.9, le Générateur peut simuler des anti-patterns de flux typiques de la littérature de Vacanti et Singh. Combinez `--mean-cycle-days` avec `--pattern`.

| Motif         | Apparence scatter plot                    | Description                                                                                                                                                                                                                 |
|---------------|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| none          | Nuage de points uniforme                  | Cycle time aléatoire sans tendance — comportement par défaut                                                                                                                                                                |
| triangle      | Triangle : angle droit en bas à droite    | Le cycle time augmente linéairement dans le temps (multiplicateur $\sim 1-4\times$ à l'intensité par défaut, plus élevé avec <code>--pattern-strength</code> ). Indique un processus qui devient progressivement plus lent. |
| flat_triangle | Triangle, augmentation s'atténue à la fin | Comme triangle mais l'augmentation est atténuée par $\tanh(2.5\times t)$ . Montre une détérioration du processus qui se stabilise.                                                                                          |
| cluster       | Groupes verticaux de points en fin de PI  | Le cycle time reste stable, mais la plupart des tickets sont livrés dans les 2 dernières semaines avant la fin du PI. Comportement orienté par délai.                                                                       |
| batch         | Colonnes de hauteur variable en fin de PI | Comme cluster mais avec cycle time très variable ( $0.1-3\times$ moyenne). Montre que tickets courts et longs sont tous poussés en fin de PI.                                                                               |

### Contrôle du cycle time

```
# Motif triangle : CT augmente de 20j à 80j
python -m testdata_generator \
  --workflow workflow.txt --issues 300 \
  --pattern triangle \
  --mean-cycle-days 20 --std-cycle-days 6 \
  --completion-rate 0.8 --seed 1 \
  --output triangle.json
```

## 5 ART\_A – Exemple complet

---

Cet exemple montre le pipeline complet du générateur au rapport final, en utilisant le projet fictif ART\_A.

### Étape 1 : Générer les données de test

```
python -m testdata_generator \  
    --workflow testdata_generator/workflow_ART_A.txt \  
    --project ART_A --issues 200 \  
    --from-date 2025-01-01 --to-date 2025-12-31 \  
    --completion-rate 0.75 --seed 42 \  
    --output ART_A_generated.json
```

### Étape 2 : Traiter avec transform\_data

```
python -m transform_data ART_A_generated.json \  
    --workflow testdata_generator/workflow_ART_A.txt
```

### Étape 3 : Créer le rapport avec build\_reports

```
python -m build_reports
```



## 5b Le scénario de portefeuille

Avec `--scenario portfolio`, le générateur crée en une seule étape un portefeuille de démonstration complet : deux solutions de trois ARTs chacune, avec tous les artefacts de la chaîne de traitement.

Dans la GUI : la section 'Demo portfolio' en bas de la fenêtre génère le scénario dans un dossier de votre choix en un clic (le champ seed est repris, 42 par défaut) ; 'Open Portfolio Report' génère le rapport de portefeuille, 'Open Delta Briefing' le briefing « qu'est-ce qui a changé ? » à partir de `snapshots/snapshot_prev/now.json` — sans ligne de commande.

```
python -m testdata_generator --scenario portfolio \
    --output demo/ --seed 42 --scale m
```

Echelles (`--scale`, aussi un selecteur dans la GUI) : les neuf registres par solution se composent d'ancres d'histoire fixes plus une population de base generee par seed — s = ancrs seules, m = demo realiste (defaut), l = test de charge (heatmap dense, matrice roadmap pleine). Les ancrs et leurs histoires sont identiques a chaque echelle ; la base respecte des invariants qui les laissent uniques (pas de second NFR viole, pas d'autre blocked en retard, pas de second SLO creve, etc.).

Profil d'ART (bouton 'ART Profiles...' pres de l'echelle, CLI `--art-profiles FICHIER.json`) : par ART, les memes reglages que pour la generation d'un ART seul peuvent etre surcharges — issues, cycle time moyen et sigma, taux de completion/to-do, backflow, motif (none/triangle/flat\_triangle/cluster/batch) avec intensite (0–100) et semaines de PI. Champs vides = defaut. Les valeurs surchargees s'appliquent aux DEUX etats du delta (prev = 88 % des issues) ; le reste garde sa valeur d'histoire. L'echelle et les profils d'ART voyagent avec le template de projet (menu Templates) et sont restaures au chargement.

Le dossier cible contient ensuite :

| Artefact          | Contenu                                                                                                                                                                                         |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| workflows/        | deux fichiers de flux de travail (étapes Alpha et Beta)                                                                                                                                         |
| raw/              | six exports JSON Jira (une exécution du générateur par ART)                                                                                                                                     |
| solutions/<name>/ | par solution : solution.json (Beta avec son propre stage_map), arts/ (IssueTimes/CFD/Transitions par ART) et registers/ avec les neuf registres sous noms standard (risks.json ... themes.json) |
| snapshots/        | snapshots de rapport pour le delta briefing (prev/now)                                                                                                                                          |
| portfolio.json    | config de portefeuille couvrant les deux solutions                                                                                                                                              |
| pi_config.json    | intervalles PI sur la fenêtre de données                                                                                                                                                        |
| README.md         | description des histoires intégrées                                                                                                                                                             |

Les données sont placées relativement à la date de génération, afin que le feu de qualité du rapport de portefeuille les évalue comme actuelles. Trois histoires sont intégrées :

#### Histoires intégrées :

- ART Alpha-3 est la valeur aberrante (cycle time environ triplé) — surligné en rouge dans le rapport de comparaison de la Solution Alpha.
- ART Beta-3 fournit des données faibles (pas de CFD, peu d'issues commencées, données vieilles de 60 jours) — confiance 'low' dans le tableau de qualité.
- Solution Beta agrège via son propre stage\_map (Vorlauf/Umsetzung/Fertig) ; la Solution Alpha utilise le chemin par défaut.
- Board ROAM : deux risques owned sont volontairement anciens (45/50 jours) — le surlignage aging se déclenche.
- NFR & runway : le NFR d'API de Beta est viole et un element runway est une lacune en retard — le dashboard montre du rouge.
- Capability map : la capacite data-insights de Beta est critique, une capacite Alpha n'a pas d'ART (uncovered).
- Heatmap des dependances : Alpha-1 → Alpha-3 bloquee et en retard ; Beta-1 → Alpha-1 en integration cross-solution.
- Log de decisions : l'hypothese ouverte de Beta a depasse sa date de revision — marquee en rouge 'review due'.
- Delta briefing : snapshots/snapshot\_prev/now.json sont fournis — --delta montre le debit, le declin de confiance de Beta-3, l'escalade d'AD-1, le nouveau risque BR-2, le fraichement en retard.
- SLO & DORA : l'API de synchronisation de Beta viole son SLO, et l'ART Beta-3 est low aussi dans l'image de livraison (CFR 38 %, coverage 31 %).
- Backlog des problemes de flux (VSC) : FP-A1/FP-B1 survivent a 3 resp. 4 conferences (rouge) ; --conference genere le dossier.
- Themes strategiques & roadmap : un theme orphelin, EP-A9 zombie ; dans le delta : EP-A9 perd son theme, EP-B2 passe P2 → P1.

Le dossier est directement utilisable : `python -m portfolio demo/portfolio.json` génère le rapport de portefeuille ; les configs de solution fonctionnent aussi individuellement. La même graine produit des données identiques le même jour.

## 6 Foire aux questions (FAQ)

---

### Différence entre données de test et données réelles ?

Pour les démos, la formation et les tests d'installation, les données de test sont le meilleur choix — sans problèmes de confidentialité, reproductibles et entièrement contrôlées. Pour les analyses de flux réelles, les données réelles reflètent l'état réel du flux de travail.

*Pour l'export manuel de données Jira réelles : Consultez le Manuel d'utilisation Get Data.*

## 7 Glossaire

---

| Terme      | Explication                                                              |
|------------|--------------------------------------------------------------------------|
| ART        | Agile Release Train — structure multi-équipes dans SAFe                  |
| API        | Application Programming Interface — interface de programmation           |
| CFD        | Cumulative Flow Diagram — comptage cumulé des tickets par étape          |
| Cycle Time | Temps depuis la première étape active jusqu'à l'étape fermée             |
| Issue      | Élément de travail dans Jira (Feature, Bug, Enabler, Story, ...)         |
| JSON       | JavaScript Object Notation — format des exports API Jira                 |
| Seed       | Valeur initiale pour le générateur aléatoire — même seed = même résultat |
| Stage      | Une étape dans le flux de travail (correspond à une colonne Jira)        |
| Workflow   | Séquence ordonnée d'étapes que parcourt un ticket                        |